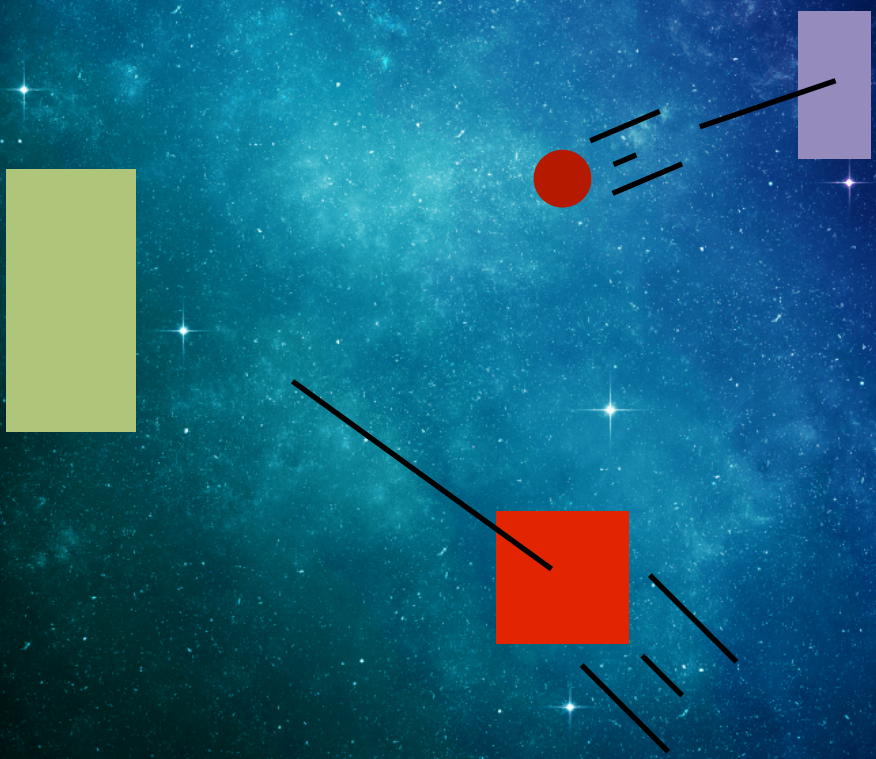
SPACE FRENZY

Proyecto de Tecnología de la Programación



DANIEL MARTÍNEZ - CABEZA DE VACA GUILLÉN

DNI: 26648686N, Asignatura: Tecnología de la Programación, Grupo: PCEO, Subgrupo: 1.4, Convocatoria: mayo 2024, Año académico: 2023/2024, Profesor: Juan Antonio Sánchez Laguna

ÍNDICE



Índice 2

Descripción de la aplicación 3

Manual de usuario 5

Organización del proyecto 6

Estructuras de datos 9

Conclusiones 23

DESCRIPCIÓN DE LA APLICACIÓN



La aplicación consiste de un videojuego, con sus correspondientes menús de inicio, de pausa y de ayuda. Para su funcionamiento se emplean distintas técnicas aprendidas a lo largo del curso de la asignatura.

Al iniciar la aplicación se presenta primeramente el menú de inicio, que muestra como imagen de fondo el escenario en el que transcurrirá la partida, por defecto este muestra el escenario importado de un fichero de texto en los archivos del proyecto. En la pantalla se muestran cuatro botones distintos que se corresponden con las siguientes funciones:

- Inicio de partida: lleva a la pantalla de juego en la que transcurre la partida.
- Menú de ayuda: lleva a una pantalla de tutorial en la que se explican brevemente los controles y objetivos del videojuego.
- Generación de mapas: sustituye el escenario actual por otro generado de forma aleatoria.
- Salida: termina la ejecución de la aplicación.

La partida comienza con el jugador en la esquina superior izquierda del escenario y durante su transcurso deberá recorrer el mapa eliminando enemigos y consiguiendo consumibles para aumentar la puntuación y conseguir mejoras como munición, vidas extra, escudos, etc.

La pantalla durante el transcurso de la partida muestra en la esquina superior derecha una representación a escala reducida del escenario en el que se muestra la posición tanto del jugador como de los enemigos, consumibles y muros. En la parte superior izquierda se muestra el numero de vidas restantes del jugador y la puntuación conseguida hasta el momento, además, el color de este recuadro indica el tipo de mejora que proporciona el consumible que hay disponible en el momento.

En cualquier momento durante la partida se puede acceder al menú de pausa, que congela la ejecución de la partida mostrando dos botones, uno que continua el transcurso de la partida y otro que vuelve a la pantalla del menú de inicio.

Al terminar la partida antes de volver al menú de inicio, se compara la puntuación obtenida con las diez mejores puntuaciones, que se almacenan en un fichero de texto. Si la puntuación actual es mayor que alguna de las del ranking, este se actualizará poniendo la puntuación de la partida en su lugar correspondiente.

MANUAL DE USUARIO



Al ejecutar la aplicación la primera pantalla, el menú de inicio permite elegir distintas opciones:

Para generar un mapa distinto al que se muestra se debe hacer clic en el botón verde situado en el lado derecho de la pantalla.

Para iniciar la partida se debe hacer clic en el botón gris de la parte superior-central de la pantalla, una vez pulsado, el juego dará comienzo.

Para acceder al tutorial donde se explican los controles básicos se debe hacer clic en el botón gris de situado en el centro de la pantalla.

Por último, pulsando el botón rojo en la parte central-inferior de la pantalla, se interrumpe la ejecución de la aplicación.

Durante la partida, se deben usar las teclas W, A, S, D para controlar el movimiento del personaje, las teclas “flecha izquierda” y “ flecha derecha” para rotar el cañón del jugador y apuntar, y la tecla “espacio” para disparar.

Si se quiere parar una partida en curso se debe pulsar la tecla “escape”, que abrirá el menú de pausa. Para reanudar la partida se debe pulsar el botón superior en pantalla o la tecla “escape” y para volver al menú de inicio y terminar la partida se debe pulsar el botón inferior.

ORGANIZACIÓN DEL PROYECTO



El proyecto está compuesto de los siguientes ficheros:

- Código:
  * **main.c**: coordina los distintos elementos de la aplicación (partida, menús, etc…).

  * **jugador.c**: implementación privada del TDA jugador.
  * **jugador.h**: interfaz pública del TDA jugador, encargado del comportamiento del personaje principal.

  * **enemigos.c**: implementación privada del TDA ejército.
  * **enemigos.h**: interfaz pública del TDA ejército, que coordina el comportamiento de el conjunto de todos los enemigos.

  * **bala.c**: implementación privada del TDA bala.
  * **bala.h**: interfaz pública del TDA bala, responsable del comportamiento de los proyectiles disparados.

  * **ráfaga.c**: implementación privada del TDA ráfaga.
  * **ráfaga.h**: interfaz pública del TDA ráfaga, encargado de coordinar el comportamiento del conjunto de todos los proyectiles.

  * **consumibles.c**: implementación privada del TDA caja.
  * **consumibles.h**: interfaz pública del TDA caja, define el comportamiento de los tesoros.

    * **camara.c**: implementación privada del TDA camara.
    * **camara.h**: interfaz pública del TDA camara que gestiona qué parte del escenario se muestra en pantalla.

    * **mapa.c**: implementación privada del TDA mapa.
    * **mapa.h**: interfaz pública del TDA mapa que define el terreno en el que se desarrolla la partida.

    * **partida.c**: implementación privada del TDA partida.
    * **partida.h**: interfaz pública del TDA partida, coordina los distintos módulos para regular el curso de la partida y los menús.

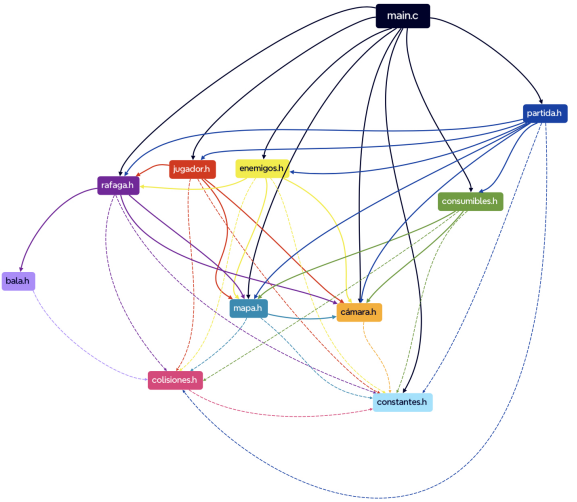
    * **colisiones.c**: implementación privada de las funciones encargadas de comprobar solape entre rectas, circunferencias, rectángulos, etc…
    * **colisiones.h**: interfaz pública de las funciones encargadas de comprobar solape entre rectas, circunferencias, rectángulos, etc…

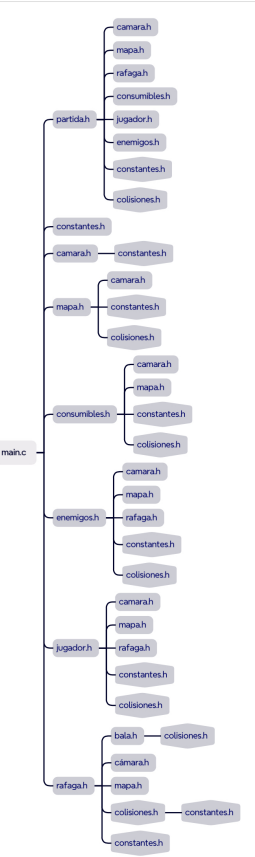
    * **constantes.h**: reúne un conjunto de constantes reutilizadas a lo largo del proyecto.

- Imágenes:
    * map_wall-tile_11zon.bpm: imagen de cada muro.
    * map2.bpm: fondo de pantalla en la partida.
    * turret.bpm: imagen del cañón del jugador.
- Ficheros de texto:
    * record.txt: almacena las mejores puntuaciones.
    * mapa.txt: guarda el mapa de la última partida jugada.
    * mapas_importar.txt: almacena el mapa que se lee al iniciar la aplicación.

DEPENDENCIAS ENTRE ARCHIVOS

Las siguientes figuras ilustradas las relaciones de dependencia entre ficheros, las flechas discontinuas y las celdas hexagonales indican una dependencia en la implementación privada mientras que las flechas continuas y las celdas rectangulares indican una dependencia pública:





ESTRUCTURAS DE DATOS



TDA JUGADOR

Nombre: Jugador.

Descripción: Representa el personaje controlado por el usuario.

Dominio de valores: El jugador tiene posición, ancho, alto, velocidad, dirección de apuntado, munición, tiempo de invencibilidad, disparo triple, enfriamiento de disparo y color.

Interfaz público:

* double get_player_pos_x(Jugador j)
  Devuelve la coordenada x de la posición de un jugador.

* double get_player_pos_y(Jugador j)
  Devuelve la coordenada y de la posición de un jugador.

* double get_player_width(Jugador j)
  Devuelve la anchura de un jugador.

* double get_player_height(Jugador j)
  Devuelve la altura de un jugador.

* double get_player_aimx(Jugador j)
  Devuelve la coordenada x de la dirección a la que apunta un jugador.

* double get_player_aimy(Jugador j)
  Devuelve la coordenada y de la dirección a la que apunta un jugador.

* int get_player_energy(Jugador j)
  Devuelve la munición restante de un jugador.

* int get_player_lives(Jugador j);
  Devuelve el número de vidas de un jugador.

* int get_player_inv(Jugador j);
  Devuelve el tiempo restante de invencibilidad de un jugador.

* int get_player_cooldown(Jugador j);
  Devuelve el tiempo restante de enfriamiento al disparar de un jugador.

* void dmg(Jugador j);
  Inflinge daño a un jugador.

* void set_inv(Jugador j, int t);
  Modifica el tiempo de invencibilidad del jugador.

* void set_energy(Jugador j, int e);
  Modifica la munición del jugador.

* void set_triple_shot(Jugador j);
  Activa el disparo triple del jugador.

* void set_lives(Jugador j, int l);
  Modifica el número de vidas del jugador.

* void color_change(Jugador j);
  Modifica el color del jugador de forma gradual.

* Jugador create_player();
  Crea e inicializa un jugador.

* void free_player(Jugador j);
  Libera un jugador.

* void draw_player(Jugador j, Camara cam);
  Dibuja un jugador respecto a una cámara.

* void move_player(Jugador j, Rafaga r, Mapa m);
  Mueve un jugador.

TDA EJÉRCITO

Nombre: Ejército.

Descripción: Representa el conjunto de todos los enemigos.

Dominio de valores: El ejército tiene un número finito de enemigos vivos, y cada enemigo tiene posición, ancho, alto, dirección, velocidad, distancia de seguimiento, tipo y enfriamiento de disparo.

Interfaz público:

* int get_army_ealive(Ejercito e);
  Devuelve el número de enemigos vivos del ejército.

* void set_ealive(Ejercito e, int n);
  Modifica el número de enemigos vivos.

* double get_enemy_x(Ejercito e, int index);
  Devuelve la coordenada x de un enemigo del ejército.

* double get_enemy_y(Ejercito e, int index);
  Devuelve la coordenada y de un enemigo del ejército.

* Ejercito create_army();
  Crea e inicializa un ejército.

* void free_army(Ejercito e);
  Libera un ejercito.

* void insert_enemy(Ejercito e, double x, double y, int type);
  Añade un enemigo a un ejército con los parametros indicados.

* void move_army(Ejercito e, double jx, double jy, int jinv, Rafaga r, Mapa m);
  Mueve el ejército.

* void draw_army(Ejercito e, Camara cam);
  Dibuja un ejército respecto a una cámara.

* int object_army_collision(Ejercito e, double x, double y, double w, double h);
  Detecta la colisión de un ejército con un rectángulo.

* int army_burst_shot_collision(Ejercito e, Rafaga r);
  Detecta el número de colisiones de un ejército con una ráfaga.

TDA BALA

Nombre: Bala.

Descripción: Representa un proyectil.

Dominio de valores: El proyectil tiene posición, dirección, velocidad y radio.

Interfaz público:

* Bala create_bullet(double x, double y, double dx, double dy, double v);
  Crea e inicializa una bala con los valores especificados.

* void free_bullet(Bala b);
  Libera una bala.

* void move_bullet(Bala b);
  Mueve una bala.

* void draw_bullet(Bala b);
  Dibuja una bala.

* double get_bullet_x(Bala b);
  Devuelve la coordenada x del centro de una bala.

* double get_bullet_y(Bala b);
  Devuelve la coordenada y del centro de una bala.

* double get_bullet_rad(Bala b);
  Devuelve el radio de una bala.

TDA RÁFAGA

Nombre: Ráfaga.

Descripción: Representa un conjunto de proyectiles.

Dominio de valores: La ráfaga tiene un número finito de elementos, cada elemento es una bala.

Interfaz público:

* Rafaga create_burst_shot();
  Crea e inicializa una ráfaga.

* void free_burst_shot(Rafaga r);
  Libera una ráfaga.

* void insert_burst(Rafaga r, Bala b);
  Añade una bala a una ráfaga.

* void move_burst_shot(Rafaga r, Mapa m);
  Mueve una ráfaga.

* void draw_burst_shot(Rafaga r, Camara cam);
  Dibuja una ráfaga.

* int burst_shot_lenght(Rafaga r);
  Devuelve la longitud de una ráfaga.

* int burst_shot_collision(Rafaga r, double x, double y, double w, double h);
  Detecta la colisión de una ráfaga con un rectángulo.

TDA CAJA

Nombre: Caja.

Descripción: Representa un tesoro consumible por el jugador.

Dominio de valores: La caja tiene posición, ancho, alto y tipo.

Interfaz público:

* int get_box_type(Caja c);
  Indica el tipo de consumible que es una caja.

* double get_box_x(Caja c);
  Devuelve la coordenada x de una caja.

* double get_box_y(Caja c);
  Devuelve la coordenada y de una caja.

* Caja create_box(Mapa m);
  Crea e inicializa un consumible.

* void free_box(Caja c);
  Libera una caja.

* int box_collision(Caja c, double x, double y, double w, double h);
  Detecta la colisión de una caja con un rectángulo.

* void move_box(Caja c, Mapa m);
  Cambia de posición una caja.

* void draw_box(Caja c, Camara cam);
  Dibuja una caja.

TDA MAPA

Nombre: Mapa.

Descripción: Representa el conjunto de todos los muros que forman el terreno de juego.

Dominio de valores: El mapa tiene un tamaño de celda y un número determinado de muros.

Interfaz público:

* Mapa create_map();
  Crea un mapa vacio.

* void free_map(Mapa m);
  Libera un mapa.

* void reset_map(Mapa m);
  Vacia un mapa.

* void save_map(Mapa m);
  Guarda un mapa en un archivo de texto.

* void read_map(Mapa m);
  Importa un mapa de un archivo de texto.

* void print_map(Mapa m);
  Imprime un mapa en la consola de comandos.

* void draw_map(Mapa m, Camara cam);
  Dibuja un mapa.

* void draw_minimap(Mapa m, int x, int y, int scale);
  Dibuja un mapa en una posición y escala determinados.

* void generate_map(Mapa m, int n);
  Genera un mapa de forma aleatoria.

* int map_collision(Mapa m, double x, double y, double w, double h);
  Comprueba la colisión de un mapa con un rectángulo.

* int line_map_collision(Mapa m, double x1, double y1, double x2, double y2);
  Comprueba la intersección de un mapa con una recta.

TDA CÁMARA

Nombre: Cámara .

Descripción: Representa una cámara que determina la parte del escenario que se muestra en pantalla.

Dominio de valores: La cámara tiene ancho, alto, posición y velocidad de seguimiento .

Interfaz público:

* double get_cam_x(Camara cam);
  Devuelve la coordenada x de la cámara.

* double get_cam_y(Camara cam);
  Devuelve la coordenada y de la cámara.

* Camara create_camera(double x, double y);
  Crea e inicializa una cámara con los valores especificados.

* void free_cam(Camara cam);
  Libera una cámara.

* void camera_follow(Camara cam, double x, double y);.
  Mueve la cámara en relación a un punto.

TDA PARTIDA

Nombre: Partida.

Descripción: Representa una instancia del juego.

Dominio de valores: Una partida puede estar activa, en menú o en pausa, ademas, cada partida tiene un número de rondas con un tiempo determinado y una puntuación total.

Interfaz público:

* int get_game_active(Partida p);
  Indica si el juego está en curso en una partida.

* int get_game_pause(Partida p);
  Devuelve el estado de pausa de la partida.

* int get_game_round(Partida p);
  Indica en qué ronda se encuentra la partida.

* int get_game_time(Partida p);
  Indica el tiempo restante hasta que empiece una nueva ronda.

* int get_game_pts(Partida p);
  Indica la puntuación conseguida.

* void add_game_pts(Partida p, int n);
  Incrementa la puntuación de una cantidad n.

* void unpause(Partida p);
  Termina el estado de pausa de la partida.

* void end_game(Partida p);
  Termina la partida actual.

* struct Leaderboard read_records();
  Lee la información del fichero record.txt, que almacena las mejores puntuaciones conseguidas.

* void update_records(Partida p);
  Compara las mejores puntuaciones con la puntuación actual y actualiza la tabla de puntuaciones acordemente.

* Partida start_game();
  Crea e inicializa una partida.

* void free_game(Partida p);
  Libera una partida.

* int check_end_game(Partida p, Jugador j);
  Indica si el juego está en curso en una partida

* void army_generation(Partida p, Ejercito e, Jugador j, Mapa m);
  Regula la generación de enemigos.

* void army_behavior(Partida p, Ejercito e, Jugador j, Rafaga rj, Rafaga re, Mapa m);
  Regula el comportamiento de los enemigos.

* void box_behavior(Partida p, Caja c, Jugador j, Mapa m);
  Regula el comportamiento del tesoro consumible.

* void draw_game(Partida p, Jugador j, Ejercito e, Rafaga rj, Rafaga re, Caja c, Mapa m, Camara cam);
  Dibuja todos los elementos visibles del juego y el HUD.

* void game(Partida p, Jugador j, Ejercito e, Rafaga rj, Rafaga re, Caja c, Mapa m, Camara cam);
  Regula el transcurso de la partida y coordina sus elementos.

* int button_click(double x, double y, double w, double h);
  Detecta cuando se hace click sobre un botón.

* int menu(Partida p);
  Gestiona el menu de inicio.

* int pause_menu(Partida p);
  Gestiona el menu de pausa.

* void ayuda(Partida p, Jugador j, Ejercito e, Rafaga r, Camara cam);
  Gestiona el menú de ayuda.

CONCLUSIONES



A la hora de realizar el proyecto las principales dificultades que he encontrado han sido relacionadas a la estructura del proyecto, ya que al implementar funciones de la aplicación que variaban de lo incluido en las prácticas guiadas a menudo he visto que era necesario reestructurar dichas partes al seguir avanzando en el proyecto ya que entraba en conflicto con la estructura contemplada en las prácticas guiadas.
Mas allá de esto, considero que el desarrollo de este proyecto no me ha causado grandes dificultades. En particular aprecio las facilidades que se han dado para desarrollar el proyecto con modificaciones respecto a lo propuesto en el boletín de prácticas.

Respecto a los contenidos de la asignatura estoy muy satisfecho con los contenidos impartidos ya que cubren las bases de la programación que se salen de los bloques básicos de iteración o la sintaxis de un lenguaje de programación.

En conjunto he disfrutado la asignatura a lo largo de su curso y considero que en cuanto a materiales provistos y a estructuración de los contenidos ha estado bien organizada.